

Séquence 3 – Parsing, structuration et traçabilité

De la ligne brute à la décision contestable

École de l'air et de l'espace

Une décision contestée doit remonter à sa source

“35,4°C dans le shelter batteries : inspection déclenchée.”

Question décisionnelle

Quel message, quelle transformation et quelles limites permettent de justifier cette décision plus tard ?

À la fin, vous saurez décider si l'indicateur est défendable

- ▶ parser sans correction silencieuse ;
- ▶ structurer sous un schéma explicite ;
- ▶ relier chaque ligne au message brut ;
- ▶ distinguer intégrité, authenticité et vérité physique ;
- ▶ traiter un quasi-doublon sans détruire la preuve ;
- ▶ qualifier confiance technique et opérationnelle.

Quatre productions rythment les 240 minutes

1. TP 1 : produire `batch001_structured.csv` ;
2. TP 2 : défendre un schéma commenté ;
3. TP 3 : rejouer le lien table $\rightarrow$ brut ;
4. incident, débat de rétention et brief décisionnel.

Chaque étape : action, confiance, preuve, incertitude, limite.

Vote initial – la décision est-elle justifiée ?

A justifiée B plausible mais non auditable

C maintenir par prudence D annuler

À noter

Confiance 0–100 %, fait disponible, preuve prioritaire, coût d'erreur.

Une chaîne de justification comporte quatre niveaux

Message brut → ligne structurée
→ indicateur → action

Chaque flèche doit nommer sa règle, sa version et sa trace.

La prudence opérationnelle ne remplace pas la preuve

Une inspection peut être rationnelle si le coût d'un faux négatif est élevé. Cela ne démontre pas que la mesure est exacte.

Deux questions distinctes

L'action est-elle prudente ? La justification data est-elle auditée ?

Le brut est conservé au plus près de la réception

- ▶ il possède déjà une enveloppe et un payload ;
- ▶ il peut contenir des métadonnées de collecte ;
- ▶ il n'est ni pur, ni automatiquement vrai ;
- ▶ il permet de reparsing avec une règle nouvelle.

Structurer, c'est projeter sous un contrat

Gain

- ▶ colonnes stables ;
- ▶ calculs ;
- ▶ contrôles.

Risque

- ▶ champs omis ;
- ▶ types forcés ;
- ▶ contexte perdu.

Question décisionnelle

Quelle contestation devient impossible si ce champ disparaît ?

La provenance relie entités, activités et responsabilités

La provenance décrit ce qui a servi, ce qui a été produit et par quelle activité.

Version S03 minimale

Fichier source, ligne, empreinte, version de schéma, code et commande.

Source : W3C, PROV-Overview et PROV-DM, 2013.

Auditabilité ne signifie pas exactitude

Auditabilité permet	Elle ne garantit pas
retrouver une source	vérité physique
rejouer une transformation	calibration
détecter un changement	identité de l'auteur
contester une règle	exhaustivité du lot

Enveloppe, payload et pipeline ont des autorités différentes

Niveau	Exemples	Question
enveloppe	topic, received_at	qu'a observé la collecte ?
payload	measured_at, value	que déclare l'émetteur ?
pipeline	source_line, hash	comment retrouver le brut ?

TP 1 – Prédire avant de parser

1. combien d'enveloppes ?
2. quels champs par niveau ?
3. que faire d'une ligne incomplète ?
4. quelle phrase défendre ?

TP 1 – Produire le CSV structuré

```
export PYTHONPATH=src
python3 -m iot_decision.traceability_cli parse \
  data/samples/batch001_messages.jsonl \
  data/processed/batch001_structured.csv
```

Attendu

15 lignes structurées ; 0 anomalie isolée.

TP 1 – Contrôler avant d'interpréter

```
head -n 3 data/processed/batch001_structured.csv  
tail -n +2 data/processed/batch001_structured.csv | wc -l
```

Attendu : en-tête complet puis 15 lignes de données.

Le mode strict rend l'échec visible

Un payload incomplet arrête le lot. Un mode tolérant doit isoler l'erreur et produire une trace.

Interdit

Remplacer silencieusement un champ obligatoire par une chaîne vide.

Point d'étape – formulation exacte

“15 enveloppes conformes au contrat s03-v1 ont été structurées.”

Non démontré

Exhaustivité, exactitude physique, authenticité et qualité des horloges.

TP 2 – Défendre le schéma champ par champ

Champ	Origine	Contrôle	Risque si supprimé
topic	enveloppe	non vide	périmètre perdu
unit	payload	vocabulaire	comparaison fausse
source_line	pipeline	entier ≥ 1	retour ambigu
schema_version	pipeline	valeur connue	règle inconnue

Pourquoi garder deux horloges ?

`measured_at` est déclaré par l'émetteur.

`received_at` est observé par la collecte.

Utilité

Comparer les deux étudie un délai apparent ; cela ne valide aucune horloge.

Source : IETF, RFC 3339, 2002.

Pourquoi message_id et source_line ?

- ▶ message_id est une identité déclarée ;
- ▶ source_line est une adresse dans le fichier ;
- ▶ aucun des deux ne prouve l'auteur ;
- ▶ leur désaccord déclenche une enquête.

Pause – 10 minutes

Au retour : que prouve vraiment une empreinte ?

Une empreinte détecte une différence avec une référence

`raw_sha256` est calculé sur le contenu exact de la ligne, sans saut final.

Elle prouve

La ligne comparée produit ou non la même empreinte.

Elle ne prouve pas

Auteur, antériorité, authenticité ou vérité physique.

Canoniser et préserver sont deux stratégies

- ▶ préserver : empreinte des octets reçus, sensible à tout changement ;
- ▶ canoniser : représentation stable avant cryptographie ;
- ▶ S03 préserve la ligne brute et documente la convention ;
- ▶ re-sérialiser JSON peut changer les octets sans changer les valeurs.

Source : IETF, RFC 8785, JSON Canonicalization Scheme, 2020.

Classement – démontré, plausible ou non démontré ?

1. le message_id est présent ;
2. le capteur nommé est l'auteur authentique ;
3. brut et CSV de référence correspondent ;
4. l'horloge capteur est exacte ;
5. la transformation peut être rejouée.

TP 3 – Vérifier tout le lot

```
python3 -m iot_decision.traceability_cli verify \  
  data/samples/batch001_messages.jsonl \  
  data/processed/batch001_structured.csv
```

Attendu

traçabilité vérifiée

TP 3 – Rejouer deux liens manuellement

1. relever fichier, ligne, message_id et empreinte ;
2. afficher la ligne JSONL ;
3. comparer les champs ;
4. écrire conclusion autorisée et interdite.

Le saut de ligne fait partie de la convention

`sha256sum` sur une sortie de `sed` inclut un saut de ligne. Le pipeline ne l'inclut pas.

Réflexe

Une divergence déclenche l'examen de la convention et de l'entrée.

Test réversible – altérez seulement une copie

Copier dans /tmp, modifier 33.8 en 33.9, relancer la vérification.

Attendu

Empreinte différente pour source_line=1. Le brut de référence reste intact.

Incident – deux messages semblent identiques

Même zone, capteur, instant mesuré, valeur, unité et séquence.

Question décisionnelle

Identiques, doublons certains, candidats doublons ou événements distincts ?

Comparer tous les champs change le diagnostic

Champ	Message A	Message B
message_id	0002	replay-a
received_at	10 :05 :00	10 :05 :02
retained	true	false
topic	/0002	/replay-a
raw_sha256	différent	différent

Un candidat doublon n'autorise pas la suppression

Une règle défendable nomme :

- ▶ clé et fenêtre de comparaison ;
- ▶ autorité qui approuve ;
- ▶ risque de faux positif ;
- ▶ sort et trace de la ligne écartée ;
- ▶ procédure de rejeu.

Mini-débat – supprimer le brut après le CSV ?

Conserver

- ▶ reparsing ;
- ▶ preuve ;
- ▶ investigation.

Supprimer

- ▶ minimisation ;
- ▶ confidentialité ;
- ▶ coût/attaque.

Produisez une politique : durée, autorité, accès, intégrité, destruction.

Le brief final tient en sept éléments

1. action et périmètre ;
2. confiance technique ;
3. confiance opérationnelle ;
4. deux preuves retrouvables ;
5. deux incertitudes ;
6. une limite ;
7. une vérification prioritaire.

150 mots maximum.

Une réponse recevable sépare les deux confiances

Structure

Correspondance brut/table vérifiée : confiance technique élevée. Décision opérationnelle : confiance moyenne au mieux, car capteur, horloge et exhaustivité ne sont pas validés.

Exit ticket – trois phrases sans écran

1. Je peux justifier... parce que...
2. Je ne peux pas affirmer...
3. Je conserverais... pendant... sous l'autorité de...

Réponse à la question directrice

Question décisionnelle

Quelles informations conserver pour justifier plus tard ?

Brut, adresse et empreinte, identifiants et contexte, deux horloges, unité, version du schéma, code/commande et journal des erreurs. **Niveau de confiance** : élevée pour la correspondance brut/table ; moyenne au mieux pour la décision opérationnelle.

Références

- ▶ W3C, *PROV-Overview* et *PROV-DM*, 2013. <https://www.w3.org/TR/prov-overview/>
- ▶ IETF, *RFC 3339 – Date and Time on the Internet*, 2002.
<https://www.rfc-editor.org/rfc/rfc3339>
- ▶ IETF, *RFC 8785 – JSON Canonicalization Scheme*, 2020.
<https://www.rfc-editor.org/rfc/rfc8785>
- ▶ Python Software Foundation, *json – JSON encoder and decoder*.
<https://docs.python.org/3/library/json.html>